

Sprawozdanie z zadania 1

Gra wyścigowa

Mikołaj Kuryło s193743
Antonina Włodarczyk s197793

1 Sformułowanie zadania i przyjęte założenia

Celem projektu było stworzenie dwuwymiarowej gry wyścigowej z widokiem prostopadłym do płaszczyzny toru. Gra miała kłaść nacisk na realistyczny (uproszczony fizycznie) model jazdy samochodem.

Do założeń szczegółowych należało:

- **Algorytm gry:** Gra opiera się na pętli sterowanej przez systemowy zegar (prywatna metoda `_tick` w klasie `Game`), która jest wywoływana z częstotliwością 60 razy na sekundę lub mniej w zależności od wydajności komputera. W każdej iteracji pętli dochodzi do aktualizacji stanu gry, w tym fizyki ruchomych i stacjonarnych obiektów, a następnie do renderowania aktualnego stanu gry na ekranie.
- **Model fizyczny:** Samochód posiada masę, rozstaw osi oraz parametry takie jak przyczepność opon zależna od podłoża. W modelu uwzględniono siły odśrodkowe, opory powietrza oraz tarcie, transfer masy między osiami będący wynikiem przyspieszenia i hamowania.
- **Grafika:** Wszystkie graficzne elementy gry (samochody, kafelki terenu) są ładowane z plików graficznych (.png) i dynamicznie obracane w zależności od kierunku poruszania się pojazdu. Kafle terenu współdzielą mechanizm renderowania z samochodem z wyjątkiem tego, że się nie przesuwają, a jedynie są renderowane w odpowiednich miejscach na mapie. Mapa budowana jest z siatki kafelków (droga, trawa, piasek), z których każdy posiada własny współczynnik tarcia wpływający na zachowanie auta.
- **Sterowanie:** Gracz steruje samochodem za pomocą klawiatury, gdzie klawisze strzałek służą do przyspieszania, hamowania i skręcania. Sterowanie jest responsywne, a reakcja samochodu na polecenia gracza jest natychmiastowa.

2 Opis rozwiązań programowych

Aplikacja została napisana w języku *Python* z wykorzystaniem biblioteki *Tkinter* do obsługi grafiki (w tym renderowania i GUI), *PIL* (Pillow) do manipulacji obrazami oraz *NumPy* do obliczeń matematycznych związanych z fizyką ruchu samochodu.

2.1 Architektura systemu

Projekt został podzielony na moduły zapewniające separację logiki od prezentacji i ułatwiające zarządzanie kodem:

- `logic.py` - rdzeń logiki gry, zawierający klasy i funkcje odpowiedzialne za fizykę, sterowanie, zarządzanie stanem gry, wczytywanie mapy i interakcje między obiektami. Implementuje dynamiczne sprawdzanie podłoża. W każdej klatce program rzutuje pozycję `x`, `y` samochodu na macierz `traction_map_matrix`. Dzięki temu współczynnik tarcia μ zmienia się płynnie między asfaltem a trawą, co bezpośrednio wpływa na sterowność.
- `renderer.py` - ze względu na ograniczenia wydajnościowe biblioteki *Tkinter*, zastosowano autorski system cache'owania tekstur w klasie `Texture`. Zamiast obracać obrazek przy każdym wywołaniu pętli, obrócone

instancje `PhotoImage` są przechowywane w słowniku, gdzie kluczem jest kąt (0-359 stopni). Redukuje to obciążenie CPU o rząd wielkości podczas intensywnych manewrów.

- `gui.py` i `menu.py` - moduły obsługujące interfejs użytkownika, okna startowe oraz wybór tekstur.
- `map_reader.py` - moduł do wczytywania mapy z pliku `.json`, który zawiera informacje o układzie kafelków terenu oraz pozycjach startowych samochodów.

2.2 Model fizyczny - fragment kodu

Poniższy fragment kodu z metody `update` klasy `CarEntity` rozszerzający podstawową klasę `Entity` pokazuje implementację modelu fizycznego samochodu, w tym obliczanie sił działających na pojazd oraz aktualizację jego pozycji i prędkości:

```
# --- Lateral forces calculations ---
# Calculate Slip Angles
# Front slip = atan(lat_vel + angular_vel * dist_to_front / long_vel) - steer_angle
if abs(self.velocity.y) > const.MIN_VELOCITY_THRESHOLD: # Avoid division by zero
    slip_angle_front = np.degrees(np.arctan2(
        self.velocity.x + self.angular_velocity * self.wheelbase *
        (1-self.mass_distribution), abs(self.velocity.y))) - self.steer_angle
    slip_angle_rear = np.degrees(np.arctan2(
        self.velocity.x - self.angular_velocity * self.wheelbase *
        self.mass_distribution, abs(self.velocity.y)))
else:
    slip_angle_front = slip_angle_rear = 0.0

# Calculate Lateral Forces
# F = -stiffness * slip_angle
force_front_lat = -self.cornering_stiffness * slip_angle_front
force_rear_lat = -self.cornering_stiffness * slip_angle_rear

# --- Longitudinal forces ---
force_front_long = accel_force * front_accel_force_long_coeff - brake_force
force_rear_long = accel_force * rear_accel_force_long_coeff - brake_force

# --- Friction circle calculations ---
# Combined tire forces per axle must stay within the friction circle.
friction_force_front = self.tire_friction_coeff * weight_front
friction_force_rear = self.tire_friction_coeff * weight_rear

input_forces_front = np.hypot(force_front_long, force_front_lat)
if input_forces_front > friction_force_front and input_forces_front > 0.0:
    scale = friction_force_front / input_forces_front
    force_front_long *= scale
    force_front_lat *= scale

input_forces_rear = np.hypot(force_rear_long, force_rear_lat)
if input_forces_rear > friction_force_rear and input_forces_rear > 0.0:
    scale = friction_force_rear / input_forces_rear
    force_rear_long *= scale
    force_rear_lat *= scale

# --- Calculate Total Forces and Resultant Acceleration ---
drag_force_lat = - const.DEFAULT_AIR_RESISTANCE_COEFF * self.velocity.x * \
    abs(self.velocity.x) - const.DEFAULT_ROLLING_RESISTANCE_COEFF * \
    self.velocity.x * self.mass
# Adjust drag based on traction
drag_force_lat /= (self.tire_friction_coeff)
total_force_lat = force_front_lat + force_rear_lat + drag_force_lat

drag_force_long = - const.DEFAULT_AIR_RESISTANCE_COEFF * self.velocity.y * \
```

```

        abs(self.velocity.y) - const.DEFAULT_ROLLING_RESISTANCE_COEFF * \
        self.velocity.y * self.mass
# Adjust drag based on traction
drag_force_long /= (self.tire_friction_coeff)
total_force_long = force_front_long + force_rear_long + drag_force_long

# --- Integrate Physics ---
# Acceleration = Force / Mass
accel_lat = total_force_lat / self.mass
self.velocity.x += accel_lat * dt

accel_long = total_force_long / self.mass
self.velocity.y += accel_long * dt

# Simple anti-overshoot rule: braking while moving forward should
# stop at zero, not instantly create reverse speed.
if self.brake > 0.0 and self.prev_velocity.y > 0.0 and self.velocity.y < 0.0:
    self.velocity.y = 0.0

# Calculate angular velocity based on longitudinal velocity and steering angle
self.speed = np.hypot(self.velocity.x, self.velocity.y)
if self.speed > const.MIN_VELOCITY_THRESHOLD:
    speed_factor = 1.0 / \
        (1.0 + const.DEFAULT_UNDERSTEER_GAIN * (self.speed**2))
    self.angular_velocity = (
        const.DEFAULT_STEERING_CORRECTION_COEFF * self.velocity.y / self.wheelbase
    ) * np.tan(np.radians(self.steer_angle)) * speed_factor
else:
    self.angular_velocity = 0.0

self.rotation += np.degrees(self.angular_velocity) * dt
self.rotation = self.rotation % 360 # Keep rotation within [0, 360)

self.velocity.x *= max(0.0, 1.0 - 4.0 * dt)

# --- Convert Local Velocity to World Position ---
rad = np.radians(self.rotation)
world_vx = np.cos(rad) * self.velocity.x - \
    np.sin(rad) * self.velocity.y
world_vy = np.sin(rad) * self.velocity.x + \
    np.cos(rad) * self.velocity.y

self.position.x += world_vx * dt
self.position.y += world_vy * dt

```

Model fizyczny bazuje na dynamice pojazdu w układzie lokalnym. Kluczowym elementem jest obliczanie kątów znoszenia opon (α), które zależą od prędkości poprzecznej pojazdu, prędkości kątowej oraz geometrii podwozia:

$$\alpha_{front} = \arctan\left(\frac{v_x + \omega \cdot a}{v_y}\right) - \delta \quad (1)$$

gdzie v to składowe prędkości, ω prędkość kątowa, a odległość osi od środka masy, a δ kąt skrętu kół.

Siły boczne są limitowane przez tzw. krąg tarcia (ang. *friction circle*), co zostało zaimplementowane poprzez skalowanie wektora sił wypadkowych, jeśli przekraczają one iloczyn współczynnika przyczepności μ i siły docisku F_z :

$$\sqrt{F_{long}^2 + F_{lat}^2} \leq \mu \cdot F_z \quad (2)$$

Dzięki temu model oddaje efekt "płużenia" auta przy zbyt gwałtownym hamowaniu w zakręcie.

3 Dyskusja osiągniętych wyników

3.1 Zalety aplikacji

- **Modułowość:** Dzięki zastosowaniu ustandaryzowanych protokołów (plik `actions.py`), logika gry jest oddzielona od implementacji graficznej, co ułatwia testowanie, rozbudowę i potencjalną zmianę biblioteki graficznej bez konieczności modyfikowania logiki gry.
- **Realistyczny model jazdy:** Implementacja uproszczonego "*kręgu tarcia*" sprawia, że samochód zachowuje się w sposób bardziej realistyczny, uwzględniając zarówno siły podłużne, jak i poprzeczne, co przekłada się na bardziej satysfakcjonujące doświadczenie z jazdy, np. auto traci sterowność podczas gwałtownego hamowania przy skręconych kołach.
- **Elastyczność map:** System wczytywania map z plików `.json` pozwala na łatwe tworzenie nowych torów wyścigowych bez konieczności modyfikowania kodu, a jedynie poprzez edycję danych mapy.
- **Stabilność numeryczna:** Zastosowanie reguły *anti-overshoot* w kodzie fizyki eliminuje drgania samochodu przy zerowych prędkościach, co jest częstym problemem w prostych symulatorach.

3.2 Ograniczenia i potencjalne ulepszenia

- **Wydażność GUI:** Zastosowanie standardowego obiektu `tk.Canvas` do renderowania całej sceny może prowadzić do spadków wydajności, zwłaszcza przy większej liczbie obiektów na ekranie. Potencjalnym ulepszeniem byłoby wykorzystanie biblioteki *Pygame* lub *PyOpenGL*, które są zoptymalizowane pod kątem renderowania grafiki 2D i 3D.
- **Brak kolizji:** Obecna implementacja uwzględnia wyłącznie interakcję samochodu z podłożem oraz wypadnięcie z mapy, ale nie implementuje kolizji z potencjalnymi przeszkodami na torze.
- **Brak modelu uszkodzeń:** Samochód nie posiada modelu uszkodzeń, co mogłoby dodać dodatkową warstwę realizmu i strategii do gry, np. poprzez zmniejszenie osiągów pojazdu po kolizji.